



Przedmiot: Uczenie Maszynowe II

System Rekomendacji Gier Steam Oparty na Analizie Treści

Wiktor Stachura
35235

Piotr Cebula
37666

11 maja 2026

Spis treści

1	Wprowadzenie	2
2	Przygotowanie Danych	2
3	Metodologia	2
3.1	Miary Podobieństwa	2
4	Wyniki i Analiza	3
5	Implementacja	3
6	Podsumowanie i Wnioski	4

1 Wprowadzenie

Niniejszy raport przedstawia przebieg realizacji projektu z zakresu systemów rekomendacyjnych. Celem projektu była implementacja modelu uczenia maszynowego zdolnego do spersonalizowanego polecenia gier wideo użytkownikom platformy Steam.

Zamiast tradycyjnego podejścia opartego o filtrowanie kolaboratywne (ang. *Collaborative Filtering*), zdecydowano się na filtrowanie oparte na zawartości (ang. *Content-Based Filtering*). System wykorzystuje historię gracza (czas spędzony w poszczególnych produkcjach pobierany przez API Steam) oraz ekstrahuje wektor cech gier z udostępnionego zbioru w celu wyliczenia ich wzajemnego podobieństwa.

2 Przygotowanie Danych

Zbiór danych pochodzi z platformy Hugging Face: `FronkonGames/steam-games-dataset`. Zestaw ten zawiera bogate metadane dla tysięcy gier, w tym recenzje, daty wydania, nazwy wydawców oraz minimalne wymagania sprzętowe.

Uzasadnienie wyboru kolumn: Do modelowania wykorzystano wyłącznie kolumny `tags` (tagi) oraz `genres` (gatunki). Odrzucono atrybuty takie jak recenzje społeczności, cena (`price`) czy nazwa dewelopera. Wybór ten podyktowany był faktem, że tagi (np. *Cyberpunk*, *RPG*, *Multiplayer*) oraz gatunki najwierniej oddają **semantyczny i mechaniczny charakter rozgrywki**. Oparcie modelu na recenzjach wymagałoby skomplikowanej analizy sentymentu (NLP), a powiązanie po deweloperach zawęziłoby rekomendacje jedynie do gier z tego samego studia (tworząc tzw. "bańkę" rekomendacyjną). Skupienie się na tagach pozwala algorytmowi parować tytuły o podobnym klimacie i mechanice, niezależnie od budżetu czy daty wydania gry.

3 Metodologia

Do ekstrakcji cech wykorzystano algorytm **TF-IDF** (ang. *Term Frequency - Inverse Document Frequency*). Model ten wybrano zamiast zaawansowanych sieci neuronowych (takich jak systemy *Two-Tower* używane przez YouTube) z następujących powodów:

1. **Efektywność obliczeniowa i pamięciowa:** Macierz TF-IDF może zostać łatwo przeliczona w pamięci RAM standardowego serwera, co pozwala na działanie w czasie rzeczywistym bez drogich kart graficznych (GPU).
2. **Problem zimnego startu (ang. *Cold-start problem*):** W przeciwieństwie do modeli opartych na filtrowaniu kolaboratywnym (np. macierzowej faktoryzacji SVD), które wymagają tysięcy ocen użytkowników dla każdej gry, metoda oparta na TF-IDF pozwala natychmiast rekomendować zupełnie nowe gry, o ile posiadają one przypisane tagi.

3.1 Miary Podobieństwa

Podstawowym algorytmem mierzącym dystans między wektorem profilu użytkownika ($\mathbf{p}_u$) a wektorem gry ($\mathbf{x}_j$) jest **podobieństwo kosinusowe** (ang. *Cosine Similarity*). Oblicza się je za pomocą wzoru:

$$\text{sim}(u, j) = \frac{\mathbf{p}_u \cdot \mathbf{x}_j}{\|\mathbf{p}_u\|_2 \|\mathbf{x}_j\|_2} \quad (1)$$

Metodę tę wybrano zamiast odległości euklidesowej, ponieważ mierzy ona kąt między wektorami, niwelując problem różnej długości dokumentów (np. gry posiadającej 20 tagów i gry z zaledwie 5 tagami).

Jako algorytm wariantowy zastosowano miarę **Jaccarda**, zdefiniowaną jako stosunek wielkości części wspólnej tagów użytkownika i gry do wielkości sumy tych zbiorów. Formalnie dla dwóch zbiorów cech A i B (np. unikalnych tagów profilu oraz tagów gry) wyraża się ją następującym wzorem:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (2)$$

4 Wyniki i Analiza

Z racji zastosowania podejścia nienadzorowanego, w którym nie dysponujemy twardymi etykietami dla nowych preferencji użytkownika, ewaluację przeprowadzono metodą analizy jakościowej otrzymywanych rekomendacji. Model skutecznie odrzuca gry już posiadane przez gracza.

W poniższej tabeli zestawiono charakterystykę dwóch wdrożonych miar podobieństwa przy testowaniu na wbudowanych profilach w aplikacji.

Tabela 1: Porównanie działania zaimplementowanych algorytmów rekomendacji

Cecha modelu	TF-IDF + Cosine (Algorytm 1)	Indeks Jaccarda (Algorytm 2)
Odporność na szum (popularne tagi)	Bardzo wysoka (dzięki wagom IDF)	Niska (każdy tag waży 1)
Złożoność obliczeniowa	Średnia (mnożenie macierzy)	Niska (operacje na zbiorach)
Subiektywna jakość rekomendacji	Wysoka (90-95%)	Przeciętna (60-70%)

Jak widać, zastosowanie wektoryzacji TF-IDF znacząco poprawia jakość rekomendacji, nadając większą wagę unikalnym cechom gier w porównaniu do prostego zliczania części wspólnej (Jaccard).

5 Implementacja

Główny moduł rekomendacyjny zaimplementowano w języku **Python** wykorzystując biblioteki **scikit-learn**, **pandas** oraz **numpy**. Aplikacja działa jako interaktywny serwis webowy, za którego obsługę odpowiada mikroframework **Flask**. Cały proces przetwarzania danych i wnioskowania analitycznego odbywa się w głównym pliku **app.py**.

Poniżej przedstawiono fragment kodu odpowiedzialny za inicjalizację obiektu wektoryzującego oraz wyuczenie modelu TF-IDF na tekstowej reprezentacji cech gier wczytanych z pliku **games.json**.

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 import pandas as pd
3
```

```

4 tfidf = TfidfVectorizer(token_pattern=r'(?u)\b\w+\b', stop_words='
    english')
5
6 if not df_games.empty:
7     tfidf_matrix = tfidf.fit_transform(df_games['features'])
8     print("Model ML gotowy!")

```

Listing 1: Definicja i wyuczenie modelu TF-IDF

Po wygenerowaniu globalnej macierzy cech system przechodzi w tryb nasłuchiwania zapytań. Gdy użytkownik wprowadzi swój identyfikator `steam_id`, aplikacja odpytuje zewnętrzne API platformy Steam w celu pobrania listy posiadanych gier oraz czasu w nich spędzonego.

Na podstawie uzyskanych danych system filtruje do 10 najdłużej ogrywanych tytułów. Wektory odpowiadające tym grom są lokalizowane w wcześniej przygotowanej macierzy `tfidf_matrix`, a następnie obliczana jest ich średnia arytmetyczna. Otrzymany w ten sposób uśredniony wektor (`user_profile`) reprezentuje wielowymiarowy profil preferencji gracza.

Kolejnym krokiem jest ewaluacja całego zbioru gier względem wyliczonego profilu. Realizowane jest to poprzez zoptymalizowaną funkcję podobieństwa kosinusowego z biblioteki `scikit-learn`.

```

1 from sklearn.metrics.pairwise import cosine_similarity
2 import numpy as np
3
4 user_indices = df_games[df_games['appid'].isin(top_appids)].index
5
6 user_profile = np.asarray(tfidf_matrix[user_indices].mean(axis=0))
7
8 similarities = cosine_similarity(user_profile, tfidf_matrix).flatten()
9 similar_indices_cosine = similarities.argsort()[::-1]

```

Listing 2: Budowa profilu użytkownika i obliczenie podobieństwa

Otrzymana po sortowaniu tablica `similar_indices_cosine` zawiera indeksy wszystkich gier z bazy uszeregowane od najlepiej do najgorzej dopasowanych.

Ostatnim etapem w potoku (ang. *pipeline*) uczenia maszynowego zaimplementowanego w systemie jest iteracyjne odfiltrowanie tych tytułów, których identyfikatory znajdują się już w bibliotece gracza. Ponadto system równolegle wykonuje obliczenia dla wariantu opartego na indeksie Jaccarda, przekształcając listy tagów na zbiory (ang. *sets*) i wyliczając stosunek ich części wspólnej do sumy, a finalne top 6 rekomendacji z obu algorytmów przekazywane jest na frontend.

6 Podsumowanie i Wnioski

Projekt zakończył się sukcesem, udowadniając skuteczność modeli przetwarzania języka naturalnego (NLP/TF-IDF) zaimplementowanych do metadanych jako bazy dla systemu rekomendacji. Podejście *Content-Based Filtering* okazało się optymalne – zarówno z perspektywy wydajności, jak i omijania problemu "zimnego startu".

Przyszłe prace nad systemem mogłyby obejmować stworzenie rozwiązania hybrydowego, które połączy aktualny model TF-IDF z filtrowaniem opartym na recenzjach graczy (*Collaborative Filtering*), wymagającym już jednak użycia bardziej zaawansowanych struktur Big Data.